

Paradigma Orientado a Objetos

Guía de ejercitación — Funcionamiento de los lenguajes orientados a objetos

Prof. Pablo Eduardo Hernández · 2026

BLOQUE 1 — La Java Virtual Machine: JDK, JRE, JVM, bytecode y Classpath

Ejercicio 1.1 · Opción múltiple

Marcá la única opción correcta en cada caso.

1. ¿Qué se necesita instalar para poder compilar código Java?

- ☐ a) El JRE, porque incluye el compilador.
- ☒ b) El JDK, porque incluye a javac y demás herramientas.
- ☐ c) La JVM, porque traduce el código fuente.
- ☐ d) Solo el Classpath.

2. El archivo que genera el compilador de Java tiene extensión:

- ☐ a) .java
- ☒ b) .class
- ☐ c) .exe
- ☐ d) .byte

3. ¿Qué componente ejecuta el bytecode en cada plataforma?

- ☐ a) El compilador javac.
- ☒ b) La JVM (Java Virtual Machine).
- ☐ c) Directamente el sistema operativo.
- ☐ d) Directamente el hardware.

4. La frase "write once, run anywhere" es posible porque:

- ☐ a) Java se compila a código de máquina específico de cada SO.
- ☒ b) El mismo bytecode lo ejecuta la JVM de cada plataforma.
- ☐ c) Java no se compila, se interpreta línea por línea desde el .java.
- ☐ d) El JRE reescribe el código fuente para cada sistema.

5. El CLASSPATH es:

- ☐ a) Una carpeta obligatoria llamada así dentro del proyecto.
- ☐ b) Una variable de entorno del SO que indica dónde buscar clases y librerías.
- ☐ c) Un método de la clase Object.
- ☐ d) Otro nombre del compilador.

Ejercicio 1.2 · Verdadero o Falso

Escribí V o F. Si es falsa, corregila.

- ☒ V ☐ F La JVM es parte del JRE.
- ☒ V ☐ F El JDK incluye al JRE.
- ☐ V ☒ F En Java se compila un binario distinto para cada sistema operativo.
- ☒ V ☐ F El compilador de Java se llama javac.
- ☐ V ☒ F El bytecode se ejecuta directamente sobre el hardware, sin intermediarios.
- ☒ V ☐ F El Garbage Collector se encarga de liberar la memoria que ya no se usa.

Ejercicio 1.3 · Relacionar columnas

Uní cada término con su descripción.

TÉRMINO	DESCRIPCIÓN	
JDK	Kit de desarrollo: compilador, generador de documentación y herramientas.	▼
JRE	Entorno de ejecución: lo mínimo para correr un programa Java ya compilado.	▼
JVM	Máquina virtual que ejecuta el bytecode sobre cada plataforma.	▼
javac	Programa que compila el código fuente a bytecode.	▼
.class	Archivo que contiene el bytecode generado.	▼
Classpath	Variable de entorno donde se buscan las clases y librerías.	▼

Ejercicio 1.4 · Ordenar el proceso

Ordená del 1 al 6 las etapas desde el código fuente hasta el hardware:

- 3 Bytecode (.class)
- 2 Compilador (javac)
- 6 Hardware
- 1 Archivo fuente (.java)
- 5 Sistema Operativo
- 4 JVM (ClassLoader, JIT, ejecución)

Ejercicio 1.5 · Respuesta breve

1. Explicá con tus palabras por qué se dice que Java es "write once, run anywhere" y en qué se diferencia de un lenguaje como C.

Significa que el mismo bytecode lo puede ejecutar la JVM de cada plataforma distinta.

2. ¿Qué problema podés tener si el CLASSPATH no está bien configurado?

Se produce un error y te va a pedir que crees la clase.

BLOQUE 2 — Estructura de un programa y convenciones de nomenclatura

Ejercicio 2.1 · Nombres autoexplicativos

Reescribí cada nombre para que sea autoexplicativo, pronunciable y con una distinción significativa. Indicá qué representaría.

int x; int edadAlumno; (porque es una variable de tipo entero)

double d; double precioProducto; (porque es una variable de tipo decimal)

boolean flag; boolean estadoActivo (porque es una variable de tipo booleano, que solo

String s2; String nombreCliente (porque es una variable de tipo cadena de caracteres q

void hacer() { ... } void calcularPromedio (porque es una variable que ejecuta metodos

Ejercicio 2.2 · camelCase vs PascalCase

Corregí cada identificador según la convención que le corresponde (clases en PascalCase; variables y métodos en lowerCamelCase).

clase: cuenta_bancaria CuentaBancaria

variable: SaldoTotal saldoTotal

método: Calcular_Interes calcularInteres

clase: factura Factura

variable: Nombre_Cliente nombreCliente

Ejercicio 2.3 · Palabras reservadas

Indicá qué líneas **NO** compilan y explicá por qué. Marcá las que tienen error:

- ☒ `int` public = 10;
- ☐ `int` contador = 0;
- ☒ `double` for = 3.14;
- ☐ `String` nombre = "Ana";
- ☒ `boolean` int = true;

Porque estan usando palabras reservadas como nombre de variables.

Ejercicio 2.4 · Estructura básica

Completá los huecos para obtener un programa que imprima "Hola mundo".

```
public      class Hola { public static      void main(String[] args) { System.out.  
println      ("Hola mundo"); } }
```

Ejercicio 2.5 · Comentarios

Agregá un comentario de bloque que describa la clase y un comentario de línea sobre el cálculo.

```
public class Circulo { double radio = 5; double area = 3.14 * radio * radio; }
```

Clase: objeto circulo.

se esta calculando el cuadrado del radio.

BLOQUE 3 — Arreglos: el arreglo como objeto, posiciones, acceso y recorrido

Ejercicio 3.1 · Declaración

Escribí la declaración y creación de cada arreglo:

1. Un arreglo de 8 números enteros.

```
int[] numeros = new int[8]
```

2. Un arreglo de 5 objetos de tipo Producto.

```
Producto[] productos = new producto[5]
```

3. Un arreglo de 3 valores booleanos.

```
boolean[] sesion = new boolean[3]
```

Ejercicio 3.2 · ¿Qué imprime?

Indicá la salida exacta de cada fragmento.

a) Fragmento 1:

```
int[] valores = new int[4]; valores[1] = 10; valores[3] = 25; for (int i = 0; i < valores.length; i++) { System.out.println(valores[i]); }
```

0 10 0 25

b) Fragmento 2:

```
String[] nombres = new String[3]; nombres[0] = "Ana"; for (int i = 0; i < nombres.length; i++) { System.out.println(nombres[i]); }
```

Ana null null

Ejercicio 3.3 · Detectá el error

Cada línea tiene un problema. Indicá cuál y por qué.

```
int[] a = new int[5]; a[5] = 3; System.out.println(a.length()); Auto[] autos = new Auto[2]; autos[0].acelerar();
```

Línea 1:

el problema es que el indice 5 no existe porque el array va de 0 a 4

Línea 2:

el parentesis que envuelve a "a.lenght()" esta de más.

Línea 3:

se debe instanciar el objeto primero antes de que ejecute el metodo.

Ejercicio 3.4 · *Escribí el código*

Dado `int[] numeros` ya cargado, escribí el fragmento que resuelve cada punto:

1. Sumar todos los elementos e imprimir el total.

```
        suma = suma + numeros[i];  
    }  
    System.out.println(suma);
```

2. Encontrar e imprimir el valor máximo.

```
        maximo = numeros[i];  
    }  
    System.out.println(maximo);
```

3. Contar cuántos elementos son mayores a 10.

```
        }  
    }  
    System.out.println(contador);
```

Ejercicio 3.5 · *Primitivos vs. referencia*

1. Indicá el valor por defecto de cada posición recién creado el arreglo:

`int[]` `0`

`double[]` `0`

`boolean[]` `True o False`

`String[]` `instancia / objeto`

2. En un arreglo de objetos, ¿qué guarda realmente cada posición: el objeto o una referencia? Explicá.

Referencia, osea, la ubicacion en memoria donde el objeto esta alojado.

Consigna

Se entrega el código fuente comentado y una breve explicación de las decisiones de diseño.

Escribí una clase `RegistroTemperaturas` que trabaje con las temperaturas de una semana (7 días) usando un arreglo de `double`. La clase debe permitir:

1. Cargar las 7 temperaturas del arreglo.
2. Calcular e imprimir el promedio de la semana.
3. Encontrar la temperatura máxima y la mínima.
4. Contar cuántos días superaron un umbral dado (por ejemplo, 30.0 grados).
5. Recorrer el arreglo usando `length` e imprimir cada día con su temperatura.

Requisitos de calidad:

- Nombres autoexplicativos y pronunciables; clase en `PascalCase`, variables y métodos en `camelCase`.
- Ningún identificador que sea palabra reservada.
- Al menos un comentario de bloque (descripción de la clase) y comentarios de línea donde aporten.
- Todo el recorrido de arreglos debe usar `length` (no números fijos).

Código alojado en el repositorio del grupo:

<https://github.com/GonzaloOrellana/TP0-P00-Grupo10>
